

Video title: Linear Regression Prediction (Module 3)

Code snippets reference

This document lists the code snippets shown in the video along with brief descriptions. We recommend reviewing this document alongside the video for the best learning experience.

Disclaimer:

The code snippets shown in this video are for conceptual illustration only. They are simplified or partial examples designed to explain the underlying ideas and may not run as-is. Please avoid copying the code directly without understanding the context and structure required to make it work.

S. No.	Code	Description
1.	<pre>import torch w = torch.tensor([2.0], requires_grad=True) b = torch.tensor([-1.0], requires_grad=True) def forward(x): return w * x + b x = torch.tensor([1.0]) y_hat = forward(x) print(y_hat)</pre>	• Slide title: Making predictions with tensors • Purpose: Demonstrates computing a prediction using the linear equation $y = wx + b$ with tensors.
2.	<pre>x = torch.tensor([[1.0], [2.0]]) y_hat = forward(x) print(y_hat)</pre>	• Slide title: Predicting for multiple inputs • Purpose: Shows how the same model can generate predictions for multiple input values at once.
3.	<pre>import torch import torch.nn as nn model = nn.Linear(1, 1) list(model.parameters()) x = torch.tensor([1.0]) y_hat = model(x) print(y_hat)</pre>	• Slide title: Using the PyTorch Linear layer • Purpose: Illustrates using the built-in <code>nn.Linear</code> layer to create a linear regression model.
4.	<pre>import torch import torch.nn as nn class LR(nn.Module):</pre>	• Slide title: Building custom modules with <code>nn.Module</code>

	<pre>def __init__(self, input_size, output_size): super(LR, self).__init__() self.linear = nn.Linear(input_size, output_size) def forward(self, x): y_hat = self.linear(x) return y_hat model = LR(1, 1) x = torch.tensor([[1.0], [2.0]]) y_hat = model(x) print(y_hat)</pre>	• Purpose: Demonstrates creating a custom model class by inheriting from nn.Module.
--	---	--